Gleipnir: Frontier-Quality AI Control Monitors at a Fraction of the Cost

Two-week sprint proposal | Jeremias Ferrao | MATS exploration phase | 19 October 2026

1 The problem: Monitoring agent actions with a minimal safety tax

Monitors are now built into widely used agent harnesses, where they score agent actions *before* they execute. This puts the monitor on the critical path of each action it checks: its latency adds to that action and its compute scales with the deployment. Frontier models monitor well but are too slow and expensive for this, imposing a large *safety tax*. Small, locally hosted monitors with frontier-level quality remove that trade-off. They also let far more researchers study and modify monitors, and fast monitoring enables new control protocols.

Gleipnir trains small open-weight monitors for scheming, sabotage, deception and other control-relevant behaviour in agent trajectories, using *binary-logit distillation*: the teacher Kimi K3's probability on the decision token (`0|1`) becomes a soft target for a LoRA student that emits that single token. On the out-of-distribution suite of Sinha et al., Gleipnir 4B is the cheapest point on the cost–performance frontier. Gleipnir 27B surpasses its teacher and nearly matches prompted Claude Opus 4.6 at ~1/50 of its cost. **The sprint brings the 4B closer to the 27B and makes the monitors deployment-ready**: measuring their weaknesses to attacks and jailbreaks so users understand their limits, validating them on untouched benchmarks and making them efficient to serve.

Monitor	pAUROC@20	$/1K evals
Gleipnir 4B (ours)	**0.782**	**0.26**
Qwen3.5-4B SFT+RL[a]	0.684	0.36
Jev 1.13	0.850	0.39
GPT-6 Luna[b]	0.837	0.84
Gleipnir 27B (ours)	**0.935**	**1.85**
Qwen3.5-27B SFT+RL[a]	0.831	2.78
Kimi K3 (our teacher)	0.908	25.12
Claude Opus 4.6, prompted[a]	0.942	94.79

Mean pAUROC@20 over six OOD sources (6,395 trajectories); uncached list price.
[a] Sinha et al.'s reported result. [b] Binary log-probs, reasoning off.

2 Sprint plan (deliverable: Gleipnir v1 model release + technical report)

Week 1: Improve on Gleipnir 4B's recipe of distilling Kimi K3's binary logits directly.

- **Teacher-assistant distillation** (Kimi K3 → Gleipnir 27B → 4B). The 27B shares the 4B's exact 248K-token vocabulary and model family, is now a stronger monitor than Kimi, and runs locally, so we can generate teacher targets cheaply for as much data as we need. It may transmit more useful information than Kimi's binary targets alone.
- **Richer supervision channels.** Soft auxiliary targets such as the *index of the harmful action*, giving the student grounded evidence about *where* the misbehaviour is, which also helps human auditors. This part stays open to other supervision ideas that emerge during the sprint.

Week 2: Find the blind spots and evaluate on benchmarks the project has never touched.

- **Adversarial attacks on the final monitors.** Two settings: *white-box*, since the released weights are public, and *black-box*, with query access only, as for a similar monitor served behind an API. The single-score interface changes the attack surface: there is no output text to steer, only one probability to lower, which a black-box attacker can query directly. Measure how easily harmful trajectories can be made to pass, and categorise the blind spots found.
- **Fresh benchmarks.** One-shot evaluation of the frozen release checkpoints on MonitoringBench and SleightBench (unseen throughout the project), with metrics fixed beforehand.
- **Release.** Model cards, weights, evaluation code and the technical report.

3 Where Neel's input would help most

- **Attack methods.** Which white-box and black-box attacks would best expose the monitors' blind spots and give results the control community finds convincing?
- **Convincing benchmarks.** Beyond MonitoringBench and SleightBench, which benchmarks would convince the community?
- **Comparison with white-box probes.** Our benchmarks use closed-model trajectories, and generating open-model data ourselves is too slow. Which AI control papers report probe scores on open-model agent trajectories we can also score?

4 Completed earlier in the exploration phase (included in the final report and release)

- **Inference optimisation.** Prefill-only B200 serving with FP4/MXFP8 kernels: up to 1.6× throughput for the 4B and 2.2–2.3× for the 27B across 4K–1M-token contexts over BF16. When a growing trajectory is re-scored after every action, prefix caching makes monitoring a 1M-token agent session 12.9× faster than re-processing the full history each time.
- **Quantisation-aware distillation (QAD)** so that low-precision deployment copies keep BF16 ranking quality.
- **Serving-time ablation of injection-triggered suspicion.** Monitors flag harmless code once monitor-directed injection or jailbreak text is present. Removing the responsible residual-stream direction at inference time cuts these false alarms to below 0.5%, with no retraining.